

Reading Query Plans

Module 9 · EXPLAIN & EXPLAIN ANALYZE

Day 2 · ~60 minutes

Module 9: Reading Query Plans

Purpose

Learn to read Yellowbrick query execution plans. Understand what the optimizer is doing, spot performance problems, and make targeted improvements to your SQL.

Main Topics

EXPLAIN · EXPLAIN ANALYZE · Plan nodes & operators
Row estimates vs actuals · DISTRIBUTE & data movement
Bloom filters & scan constraints · Red flags & query rewrites

Yellowbrick
Business Analysts

Day 2 - Afternoon

~60 minutes

[samples/retail/](#)

How Yellowbrick Executes a Query



What Does EXPLAIN Actually Do?

✓ What it DOES

- Shows the optimizer's execution strategy
- Estimates rows at each step
- Identifies operations: scans, joins, sorts, aggregations
- Reveals data distribution and movement
- Returns immediately — no data touched

✗ What it DOES NOT do

- Does not execute the query
- Does not scan or read table data
- Does not return actual row counts
- Does not show real runtime or I/O
(use EXPLAIN ANALYZE for that)

**Think of EXPLAIN as asking "How are you planning to do this?" before the work begins.
It's free — run it on every slow query before you touch anything.**

EXPLAIN Syntax

Syntax 1 · Estimate the plan (no data touched)

```
EXPLAIN
SELECT s.store_name, SUM(f.total) AS total_revenue
FROM   samples.retail.fact_sales    f
JOIN   samples.retail.dim_store    s ON f.store_key = s.store_key
GROUP BY s.store_name
ORDER BY total_revenue DESC;
```

Syntax 2 · Run the query AND show the plan (EXPLAIN ANALYZE)

```
EXPLAIN ANALYZE
SELECT s.store_name, SUM(f.total) AS total_revenue
FROM   samples.retail.fact_sales    f
JOIN   samples.retail.dim_store    s ON f.store_key = s.store_key
GROUP BY s.store_name
ORDER BY total_revenue DESC;
```

Simply prefix any SELECT with EXPLAIN or EXPLAIN ANALYZE — the rest of your SQL stays the same.

Reading the Output: The 4 Columns

id	rows_planned	workers	node
id Step number. The order in which nodes appear in the plan tree. NOT execution order.	rows_planned The optimizer's estimate of how many rows this step will process. Higher = more work.	workers "all" = runs on every compute worker in parallel (fast). "single" = runs on one worker.	node The type of operation: SCAN, JOIN, GROUP BY, SORT, DISTRIBUTE, FILTER, etc.

Reading the Plan: Always Bottom to Top

1 **SELECT**

▲ Final result sent to you

2 `DISTRIBUTE SORT`

▲ Sort result on the lead worker

3 `GROUP BY`

▲ Final aggregation per group

4 `DISTRIBUTE ON HASH`

▲ Shuffle data to co-locate groups

5 `GROUP BY PARTIAL`

▲ Pre-aggregate locally on each worker

6 **SCAN fact_sales**

Read the table — this is WHERE WE START

Data flows upward. Start reading from the bottom (SCAN = get data), work up to the top (SELECT = result).

Node Types: A Quick Reference

SCAN

Reads a table from the column store. Look for `scan_constraints` and `bloom()` — these are signs of efficient filtering.

GROUP BY

Performs aggregation (SUM, COUNT, AVG). A PARTIAL version runs on each worker first; the final GROUP BY combines results.

DISTRIBUTE ON HASH

Sends rows to workers based on a hash of a column value. Used to co-locate rows with the same key before joining or grouping.

DISTRIBUTE REPLICATE

Sends a full copy of a small table to every worker. Common for small dimension tables in star schema joins.

DISTRIBUTE SINGLE

Funnels all rows to one worker. Watch for this on large tables — it eliminates parallelism and is a potential bottleneck.

INNER HASH JOIN

Builds a hash table from one input, then probes it with the other. The |-BUILD side is the smaller (inner) table.

SORT

Orders rows. Usually appears before DISTRIBUTE SORT when results need a final ORDER BY. Sorting large sets is expensive.

FILTER

Applies a WHERE or HAVING condition after another node. If a filter appears above a SCAN (not inside it), data was scanned unnecessarily.

Example 1 — Aggregation with HAVING

The Query

```
EXPLAIN
SELECT  store_key
        , COUNT(*) AS txn_count
FROM    samples.retail.fact_sales
GROUP BY store_key
HAVING  COUNT(*) > 50000;
```

The Plan Output

id	rows_planned	workers	node
9	34	all	SELECT
1	34	all	FILTER (COUNT > 50000)
2	101	all	GROUP BY (store_key)
3	101	all	DISTRIBUTE ON HASH(store_key)
4	101	all	GROUP BY PARTIAL (store_key)
6	5,493,074	all	SCAN fact_sales

Step 6 — SCAN

Reads all 5.49M rows. No scan_constraints visible — this is a full table scan, which is expected here since no row ordering key is being filtered.

Steps 4 + 2 — Two-Phase GROUP BY

GROUP BY PARTIAL first reduces data locally on each worker, then DISTRIBUTE shuffles groups together for the final GROUP BY. This is Yellowbrick's efficient parallel aggregation pattern.

Step 1 — FILTER

The HAVING clause appears as a FILTER node above the GROUP BY — it's applied after aggregation, not during the scan. This is correct and expected.

Example 2 — Star Schema Join

```
EXPLAIN
SELECT s.store_name, SUM(f.total) AS total_revenue
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_store s ON f.store_key = s.store_key
GROUP BY s.store_name
ORDER BY total_revenue DESC;
```

id	rows_planned	workers	node
19	98	single	SELECT
2	98	all	DISTRIBUTE SORT
7	98	all	GROUP BY PARTIAL (s.store_name)
8	5,493,074	all	INNER HASH JOIN ON (s.store_key = f.store_key)
10	5,493,074	all	-SCAN fact_sales AS f bloom(store_key) scan_constraints: min_max(store_key)
13	100	all	-BUILD HASH
20	100	all	DISTRIBUTE REPLICATE
16	100	all	SCAN dim_store AS s

- **DISTRIBUTE REPLICATE**: dim_store has only 100 rows — Yellowbrick sends a full copy to every worker. No data movement at join time.
- **SELECT = "single"**: Final ORDER BY collects results on one worker. Only 98 result rows move — not the raw 5.49M rows.
- **bloom(store_key)**: Bloom filter on the SCAN skips fact_sales rows that can't possibly match dim_store keys — before the join runs.

Part 3

EXPLAIN ANALYZE

Run the query. See the truth.

EXPLAIN vs. EXPLAIN ANALYZE

	EXPLAIN	EXPLAIN ANALYZE
Runs the query?	No	Yes — query executes fully
rows_planned column	Yes	Yes
rows_actual column	No	Yes — real rows processed
skew column	No	Yes — data balance across workers
Execution time	No	Yes — planning + execution timing
I/O stats (Read/Net)	No	Yes — bytes read, distributed, network
query_id returned	No	Yes — look up in sys.log_query later
When to use	Safe to run on any query anytime — zero cost	Use when diagnosing a problem or slow query

New Columns You Get with ANALYZE

```
id    rows_planned  rows_actual  skew  workers  node
9      5,493,074    5,493,074    0.00   all     SCAN fact_sales
```

rows_actual

The real number of rows that passed through this node when the query ran. Compare to rows_planned to spot estimation errors.

skew

How unevenly data was distributed across workers. 0.00 = perfect balance. Values > 0.30 indicate some workers did much more work than others.

read_efficiency

% of column-store data that was useful (in SCAN nodes). High values (>70%) mean good column and row pruning. Low values mean extra I/O.

Execution time / query_id

Total elapsed time for the query. query_id lets you look this query up later in sys.log_query for historical analysis.

Example 3 — A Healthy Plan (rows_planned ≈ rows_actual)

```
EXPLAIN ANALYZE
SELECT s.store_name, SUM(f.total) AS total_revenue
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_store s ON f.store_key = s.store_key
GROUP BY s.store_name
ORDER BY total_revenue DESC;
```

id	rows_planned	rows_actual	skew	workers	node
19	98	100	0.00	single	SELECT
2	98	100	0.00	all	DISTRIBUTE SORT
8	5,493,074	5,493,074	0.00	all	INNER HASH JOIN
10	5,493,074	5,493,074	0.00	all	-SCAN fact_sales read_efficiency=62.4%
13	100	100	0.00	all	-BUILD HASH
16	100	100	0.00	all	SCAN dim_store read_efficiency=0.0%

Read: 24.14 MiB · Execution time: 1,848 ms · query_id: 1203734

- rows_planned ≈ rows_actual throughout — the optimizer had accurate statistics. This is a well-calibrated plan.
- skew = 0.00 on all nodes — data was perfectly balanced across workers. No bottlenecks.
- read_efficiency 62.4% on fact_sales shows column pruning at work. 0.0% on dim_store is normal (tiny table).

Example 4 — A Row Count Mismatch (Problem!)

```
EXPLAIN ANALYZE
SELECT  c.membership_tier, COUNT(*), AVG(f.total)
FROM    samples.retail.fact_sales f
JOIN    samples.retail.dim_customer c ON f.customer_key = c.customer_key
WHERE   f.total > 500
GROUP  BY c.membership_tier;
```

id	rows_planned	rows_actual	skew	workers	node
...					
8	2,746,537	4,297,479	0.00	all	INNER HASH JOIN
10	2,746,537	4,297,479	0.00	all	-SCAN fact_sales f
					(f.total > 500)
					read_efficiency=56.3%
13	99,994	99,994	0.00	all	-BUILD HASH
16	99,994	99,994	0.00	all	SCAN dim_customer c

⚠ The Mismatch

The planner estimated 2,746,537 rows after WHERE total > 500, but 4,297,479 rows actually passed — 57% more than expected. This means the table statistics are stale. The optimizer assumed ~50% of rows satisfy the filter, but in reality ~78% do. Run ANALYZE on the table to refresh statistics.

I/O Stats — What Did the Query Actually Read?

```
-- Summary line at the bottom of every EXPLAIN ANALYZE output:  
Read: 24.14 MiB · Distributed: 4.29 KiB · Network: 0.00 B  
Execution time: 1,848 ms  
  
-- Per-node detail on SCAN nodes:  
read_efficiency=62.4%, rows_from_column_store=5,493,074, rows_from_row_store=0
```

Read

Total bytes read from storage. Compare this across query rewrites — if it drops, you're doing less I/O, which usually means faster.

Distributed

Bytes shuffled between workers. Large values mean expensive DISTRIBUTE HASH operations. Ideal: keep joins on co-located data.

Network

Cross-node network traffic. Should typically be near 0 for well-designed queries. High values indicate poor distribution strategy.

read_efficiency

% of column data that was actually useful. 80%+ is good. Low values (< 30%) often mean queries need better filters or ordering keys.

When to Use EXPLAIN vs. EXPLAIN ANALYZE

EXPLAIN

- ✓ Writing a new query — check the plan before running
- ✓ Prototyping a complex join or aggregation
- ✓ Comparing two query approaches without waiting
- ✓ The query takes a long time and you want to investigate safely
- ✓ Teaching or learning — zero cost, no impact on the system

EXPLAIN ANALYZE

- ✓ A query ran slow and you want real execution stats
- ✓ You suspect the optimizer's row estimates are wrong
- ✓ You need actual timing to compare before/after a rewrite
- ✓ You want the query_id for deeper investigation in `sys.log_query`
- ✓ You're confirming a fix actually improved performance

Part 4

What the Plan Is Telling You

Interpreting operators, distribution, and data movement

Workers: "all" vs. "single"

workers = "all"

What it means: Every compute worker runs this operation simultaneously.

Good news: This is fully parallel — Yellowbrick is using all available compute capacity.

Most of your plan nodes should show "all". The more "all" nodes, the more parallel your query is.

```
10    5,493,074    all    SCAN fact_sales
```

workers = "single"

What it means: This operation runs on one worker only. Everything funnels to a single point.

Expected for: The final SELECT node (result delivery) and ORDER BY on aggregated results — both are normal.

⚠ Watch for: SCAN or JOIN nodes with "single" on large tables. This means your data is NOT being processed in parallel.

```
19    98    single    SELECT ← normal
```

DISTRIBUTE Nodes: Why Data Movement Costs Time

A DISTRIBUTE node means rows are being sent from some workers to others. This takes network time and CPU. Minimize unnecessary DISTRIBUTE operations for faster queries.

DISTRIBUTE ON HASH(column)

When: Before a GROUP BY or JOIN when rows need to be co-located by key value.

Enables parallel, co-located joins and aggregations. This is expected and efficient.

DISTRIBUTE REPLICATE

When: When one input to a join is small enough to copy everywhere (dimension tables).

Eliminates the need to move the large fact table. Very efficient for star schema queries.

DISTRIBUTE SORT

When: At the end of a query to collect sorted results on the final worker.

Only happens when your query has ORDER BY. Only the final result rows move — not raw data.

DISTRIBUTE SINGLE

When: Forces all rows to a single worker. Used for LIMIT and some global sorts.

⚠ Watch for this on large data sets. If rows_planned is high here, parallelism is lost.

Bloom Filters & Scan Constraints — Good Signs to Spot

```
-- These annotations appear under a SCAN node when Yellowbrick is being smart:
```

```
10    5,493,074    all    |-SCAN fact_sales AS f
                        |   f.store_key = bloom(13)
                        |   scan_constraints: min_max(f.store_key)
```

bloom(column) — Pre-join filter

Before the join even runs, Yellowbrick builds a tiny in-memory "membership set" from the small dimension table. When scanning the large fact table, rows that can't possibly match are skipped immediately — without reading their full data.

The number (e.g., bloom(13)) is the size of the filter in kilobytes. Larger = more distinct join keys.

scan_constraints: min_max(column) — Row range pruning

When a table has an ordering key (sort key), Yellowbrick stores min/max values per storage segment. A WHERE clause or join condition on that key lets Yellowbrick skip entire segments of the table without reading them.

Presence of scan_constraints is a sign your query is well-aligned with how the table is ordered.

If you see neither bloom() nor scan_constraints on a large SCAN — the query is doing a full table read. That's the first place to investigate.

Skew — When Some Workers Work Much Harder

```
-- skew column only appears in EXPLAIN ANALYZE output
```

id	rows_planned	rows_actual	skew	workers	node
6	5,493,074	5,493,074	0.00	all	SCAN fact_sales ← Balanced
5	823,960	823,960	0.52	all	GROUP BY (category) ← Skewed!

What is skew?

In a distributed system, each worker should process roughly equal amounts of data. Skew occurs when some workers receive far more rows than others — because the data is unevenly distributed.

Skew = 0.00 means perfect balance. Skew > 0.30 is worth investigating. High skew means the whole query waits for the slowest worker.

Why does skew happen?

Most common cause: distributing on a column with low cardinality (few distinct values). For example, if you GROUP BY country and 80% of your rows are "USA" — one worker gets 80% of the work.

Also caused by: distributing on a column with NULL values (all NULLs go to one worker), or heavy skew in the underlying data.

When you see high skew, bring it to your DBA — it usually requires a table redesign to fix properly.

Red Flags: What to Watch for in a Plan

⚠️ **SCAN with no scan_constraints or bloom()**

No ordering key in the filter — all rows are being read. Check if the table has a sort key and if your WHERE clause uses it.

⚠️ **DISTRIBUTE SINGLE on a large table**

All data funnels to one worker, eliminating parallelism. Usually caused by missing ORDER BY co-location or non-parallel operations.

⚠️ **rows_actual >> rows_planned (EXPLAIN ANALYZE)**

Stale table statistics. The optimizer underestimated data. Ask your DBA to run ANALYZE on the affected tables.

⚠️ **skew > 0.30 on a GROUP BY or JOIN node**

Data distribution is uneven — some workers wait idle while others are overloaded. May require a redistribution key change.

⚠️ **FILTER node above a SCAN (not inside it)**

A WHERE clause wasn't pushed down into the scan. This means rows were read and then discarded. Rewrite to avoid expressions on columns (e.g., UPPER()).

⚠️ **Unexpectedly high rows_planned on a SCAN**

The optimizer expects to read far more rows than you expect. Check if you're missing a join condition or WHERE clause.

Part 5

Writing Better Queries

Using EXPLAIN to guide query improvements

Before / After A — Avoid Functions on Filtered Columns

BEFORE

```
SELECT p.category, SUM(f.total)
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_product p ON
f.product_key = p.product_key
WHERE  UPPER(p.category) = 'ELECTRONICS'
GROUP BY p.category;
```

```
-- SCAN dim_product has NO scan_constraints:
13  2,381  all  SCAN dim_product AS p
      UPPER(p.category) = $0
```

UPPER() wraps the column — Yellowbrick cannot use its scan ordering optimization. Every row must be read and tested.

AFTER

```
SELECT p.category, SUM(f.total)
FROM   samples.retail.fact_sales f
JOIN   samples.retail.dim_product p ON
f.product_key = p.product_key
WHERE  p.category = 'Electronics'
GROUP BY p.category;
```

```
-- SCAN dim_product NOW has scan_constraints:
13  2,500  all  SCAN dim_product AS p
      p.category = $0
      scan_constraints:
(dim_product.category = $0)
```

Direct comparison without wrapping the column — Yellowbrick can use min/max constraints to skip segments. Fewer rows read, faster scan.

Rule: Never wrap a column in a function on the filter side (UPPER, LOWER, CAST, TRIM, etc.). Instead, store data consistently and filter without transformation.

Before / After B — SELECT * vs. Targeted Columns

BEFORE

```
-- Returns all 20+ columns from a 5M-row table
SELECT *
FROM   samples.retail.vw_sales_detail
WHERE  year = 2024
      AND state = 'CA';
```

```
-- EXPLAIN ANALYZE shows:
SCAN fact_sales  read_efficiency=21.8%
rows_from_column_store=5,493,074
```

Low read_efficiency (21.8%) — Yellowbrick reads all columns but your query only needs a few. In a columnar database, each unused column is wasted I/O.

AFTER

```
-- Request only what you actually need
SELECT  sale_date, store_name, city
        , product_name, category
        , quantity, total
FROM    samples.retail.vw_sales_detail
WHERE   year = 2024
```

```
-- EXPLAIN ANALYZE now shows:
SCAN fact_sales  read_efficiency=68.4%
rows_from_column_store=5,493,074
```

read_efficiency jumps to 68.4% — Yellowbrick now skips the unused columns entirely (amounts to less physical I/O from the column store).

Columnar storage tip: Only read what you need. Every extra column in SELECT * means more data read from disk.

The EXPLAIN Workflow: Read plan → identify bottleneck → rewrite → re-EXPLAIN → confirm improvement

The 5-Step EXPLAIN Workflow

1

Run EXPLAIN on your query

Add EXPLAIN before any SELECT. It costs nothing — no data touched, no wait time.

2

Find the largest SCAN node

Look at rows_planned. The biggest number is usually where the work begins.

3

Check for scan_constraints & bloom()

If missing on a large scan, the optimizer can't skip data. Review your WHERE clause and join keys.

4

Spot suspicious DISTRIBUTE or "single" nodes

DISTRIBUTE SINGLE on large data is a warning sign. Large DISTRIBUTE ON HASH may indicate no co-location.

5

Rewrite and re-EXPLAIN

Make one change, re-run EXPLAIN, compare. If EXPLAIN ANALYZE shows rows_actual $\neq$ rows_planned, ask for ANALYZE on the table.

Module 9 — Key Takeaways

- 1 EXPLAIN shows the query plan — free, instant, no data touched. Use it every time you write a complex query.
- 2 EXPLAIN ANALYZE runs the query and adds rows_actual, skew, I/O stats, and timing. Use it to diagnose a slow query.
- 3 Read plans bottom to top. The SCAN is where data enters; the SELECT is where results leave.
- 4 Look for bloom() and scan_constraints on SCAN nodes — they mean Yellowbrick is efficiently skipping data.
- 5 Watch for DISTRIBUTE SINGLE on large tables, skew > 0.30, and rows_actual >> rows_planned as red flags.
- 6 Avoid wrapping columns in functions (UPPER, CAST) on the filter side — it defeats scan optimizations.

Reference: docs.yellowbrick.com/7.4.3/ybd_sqlref/explain.html